



Ingeniería informática
Bases de datos distribuida
Fernando Estrella

Proyecto final

Joffre Verdezoto – Raúl Luna

26 / 7 / 2026

Índice

Proyecto final.....	5
1. Introducción.....	5
2. Objetivos.....	5
2.1. Objetivo general.....	5
2.2. Objetivos específicos.....	5
3. Descripción del caso de uso.....	6
3.1. Caso de uso seleccionado.....	6
3.2. Adaptación del caso de estudio a una pokedex.....	6
3.3. Justificación de la solución propuesta.....	6
3.4. Correspondencia entre inventario básico y pokédex.....	6
4. Requisitos del sistema.....	7
4.1. Requisitos funcionales.....	7
4.2. Requisitos no funcionales.....	8
5. Diseño y arquitectura del sistema.....	8
5.1. Arquitectura general del sistema.....	9
5.2. Diagrama de componentes.....	10
5.3. Flujo del sistema.....	11
5.4. Modelo de datos.....	12
5.5. Reglas de transformación y sincronización.....	13
6. Implementación del sistema.....	15
6.1. Tecnologías utilizadas.....	15
6.2. Configuración del entorno.....	16
6.3. Implementación de la API Rest.....	17
6.4. Descripción de la implementación del código.....	17
6.5. Pruebas del sistema.....	19
6.6. Resultados esperados y obtenidos.....	22
6.7. Limitaciones y trabajo futuro.....	24
7. Conclusiones.....	24
8. Recomendaciones.....	25
9. Bibliografía.....	25
10. Anexos.....	26
10.1. Anexo 1: Configurar el entorno.....	26
10.2. Anexo 2: Códigos relevantes.....	27
10.3. Scripts.....	35
10.4. Evidencias adicionales.....	35

Proyecto final

1. Introducción.

En el presente desarrollar aplicaciones requiere de sistemas que sean capaces de gestionar inmensas cantidades de información, por ello se utilizan diferentes tecnologías de almacenamiento, en algunos casos se utiliza arquitecturas tradicionales que se basan en un único motor de base de datos, y esto presenta limitaciones en casos en los que se requiera combinar información estructurada, operaciones transaccionales y datos que sean flexibles por ende se plantea el desarrollo de una aplicación web distribuida basada en una pokédex, la cual busca imitar la integración de bases de datos relacionales y NoSQL dentro de una misma arquitectura tecnológica (BDD heterogénea), para este caso es utilizada la base de datos postgresSQL como sistema principal para administrar información estructurada de Pokémon y entrenadores, garantizando consistencia mediante tablas, relaciones y restricciones de integridad, y por otra parte se utiliza mongoDB la cual nos permite almacenar información documental mediante estructuras JSON, lo cual nos proporciona mayor flexibilidad para datos adicionales y registros generados durante la sincronización.

2. Objetivos.

2.1. Objetivo general.

Diseñar e implementar una aplicación web distribuida donde se integre postgresSQL y mongoDB como bases de datos, con la finalidad de manejar la información de pokemones y entrenadores mediante operaciones CRUD, autenticación de usuarios y mecanismos de transformación y sincronización de datos entre las bases de datos.

2.2. Objetivos específicos.

- Diseñar un modelo de datos relacional en postgresSQL y un modelo documental en mongoDB que permitan almacenar y gestionar la información
- Desarrollar una aplicación web utilizando react (Frontend) y django (Backend), permitiendo la interacción con los servicios del sistema mediante una arquitectura cliente-servidor.
- Configurar la comunicación entre django (Backend), postgresSQL y mongoDB con mecanismos que permitan almacenar, consultar y transformar información entre ambas bases de datos.
- Implementar reglas de transformación de datos que permitan la conversión de los registros almacenados en postgresSQL en documentos que sean compatibles

con mongoDB, y se mantenga la consistencia de la información durante el proceso.

- Desarrollar mecanismos de sincronización de datos entre postgresQL y mongoDB.

3. Descripción del caso de uso.

3.1. Caso de uso seleccionado.

Se desarrolla una pokédex funcional como un sistema de inventario para administrar información de pokémon, y entrenadores, donde la aplicación permite la creación, consulta, modificación, y eliminación de registros de manera sencilla, el sistema hace uso de dos bases de datos, postgresQL para guardar datos organizados, y mongoDB para almacenar información con mayor flexibilidad, logrando que ambas bases de datos trabajen juntas dentro del sistema para simular una BDD.

3.2. Adaptación del caso de estudio a una pokédex.

El proyecto transforma la idea de un inventario en una aplicación pokédex donde se administra tanto pokemon como entrenador, cada uno de los pokemones cuenta con características que permiten identificarlo (Nombre, tipo, altura, peso e imagen), asimismo los entrenadores cuentan con características que permiten su identificación, y lo organiza dentro del sistema, por ello la aplicación mantiene un control ordenado de los datos.

3.3. Justificación de la solución propuesta.

La implementación de una pokédex como caso de estudio permite analizar una arquitectura híbrida basada tanto en base de datos postgresQL y mongoDB, donde la una se encarga de gestionar la información estructurada garantizando integridad y consistencia, mientras que la otra es complementaria para que el sistema tenga un almacenamiento flexible, conjunto con ello la interacción entre frontend, backend, y bases de datos, permite observar los procesos de integración, de transformación, y de sincronización dentro de un entorno distribuido.

3.4. Correspondencia entre inventario básico y pokédex.

SISTEMA DE INVENTARIO	SISTEMA POKÉDEX
Producto	Pokémon.
Código del producto	Identificador del pokémon.

Categoría	Tipo del pokémon.
Características del producto	Peso, altura e imagen.
Usuario administrador	Usuario autenticado.
Gestión de elementos	CRUD de pokemones y entrenadores.
Registro de cambios	Logs de sincronización.

4. Requisitos del sistema.

4.1. Requisitos funcionales.

- **Inicio de sesión de usuarios:** La aplicación debe contar con un inicio de sesión que permita identificar usuarios, y controlar quiénes pueden acceder a las opciones de administración del sistema.
- **Control de acceso:** La aplicación debe reconocer usuarios registrados, y los que sean visitantes, logrando así modificar la información sólo a quienes hayan iniciado sesión para mantener un mejor control.
- **Gestión de pokémon:** La aplicación debe permitir gestionar los datos de los pokémon, con operaciones CRUD para agregar, revisar, modificar, y eliminar información cuando el usuario necesite.
- **Gestión de entrenadores:** La aplicación debe permitir gestionar los datos de los entrenadores, con operaciones CRUD para agregar, revisar, modificar, y eliminar información cuando el usuario necesite.
- **Almacenamiento en postgresSQL:** Se deben guardar datos organizados tanto de pokemones, como de entrenadores en postgresSQL, utilizandola como base de datos principal del sistema.
- **Integración con mongoDB:** Debe conectarse con mongoDB para guardar información adicional en formato documental, permitiendo el almacenamiento de datos de manera flexible, y organizada.
- **Transformación de datos:** El sistema debe aplicar los debidos procesos de transformación que se definen para convertir registros de postgresSQL en documentos compatibles con mongoDB, para asegurar la correcta adición de los datos.
- **Sincronización de datos:** La aplicación debe permitir sincronizar información entre postgresSQL, y mongoDB.

- **Registro de sincronizaciones:** La aplicación debe guardar detalles de cada vez que se realice una sincronización, con fecha, con las bases involucradas, la cantidad de datos transferidos, y el resultado obtenido.
- **API REST:** El backend debe conectar la interfaz de usuario con las funciones internas del sistema, logrando la interacción eficiente, y correcta con la aplicación.

4.2. Requisitos no funcionales.

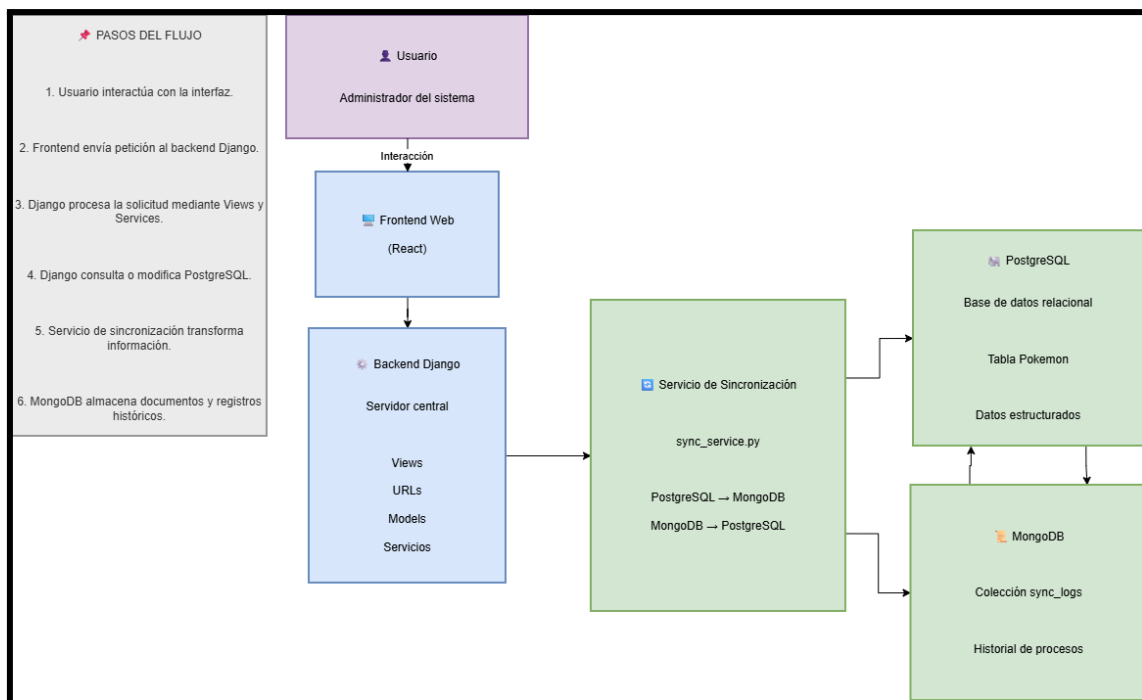
- **Seguridad:** Se debe garantizar la seguridad de las operaciones administrativas mediante mecanismos de autenticación, y el control de permisos para los usuarios.
 - **Integridad de datos:** Se debe garantizar la consistencia de la información almacenada entre las bases de datos, y evitar diferencias para asegurar la gestión de datos.
 - **Escalabilidad:** El sistema debe permitir agregar nuevos datos, o funciones de forma sencilla, para mantener su funcionamiento actual sin generar problemas en el sistema.
 - **Usabilidad:** Interfaz sencilla de usar, y capaz de recibir visitantes, y usuarios registrados que puedan navegar, y realizar acciones según sus permisos.
 - **Compatibilidad:** El sistema debe garantizar la comunicación eficiente entre react (Frontend), django (Backend), postgresQL (BD), y mongoDB (BD), permitiendo la integración adecuada de los componentes.
- 5. Diseño y arquitectura del sistema.**

5.1. Arquitectura general del sistema.

- 5.1.1. Cliente - servidor:** La arquitectura general se basa en cliente-servidor que separa la interfaz de usuario y la lógica del sistema, react encargándose de la parte de la presentación, y django encargándose de las solicitudes, reglas de negocio, y comunicación con postgresSQL, y mongoDB mediante tecnologías compatibles, de esta forma con este modelo se facilita la integración de componentes, la organización del procesamiento, y el mantenimiento de la solución.
- 5.1.2. Componentes (Frontend):** El frontend desarrollado con react representa la capa de presentación de la aplicación, es una interfaz dinámica para la interacción del usuario, sus funciones incluyen la visualización de información, la gestión de formularios, la navegación y el control de opciones según autenticación, y comunica los resultados de las operaciones realizadas, ocultando la complejidad interna del backend, y el almacenamiento.
- 5.1.3. Componentes (Backend):** El backend con django maneja la lógica de negocio, y la comunicación con servicios externos, hace uso de modelos para gestionar estructuras de datos, vistas para procesar solicitudes y rutas para organizar accesos, asimismo se encarga de incorporar servicios de sincronización, como la conexión con mongoDB y la transformación de registros relacionales de postgresSQL hacia documentos de formato JSON.
- 5.1.4. Bases de datos (PostgreSQL):** Base principal donde se guarda la información organizada del sistema, con la capacidad de almacenar datos de pokémon, entrenadores y usuarios registrados, incluso permite mantener la información correcta mediante relaciones y reglas de seguridad, asegurando los datos que permanezcan ordenados, y consistentes durante el funcionamiento de la aplicación.

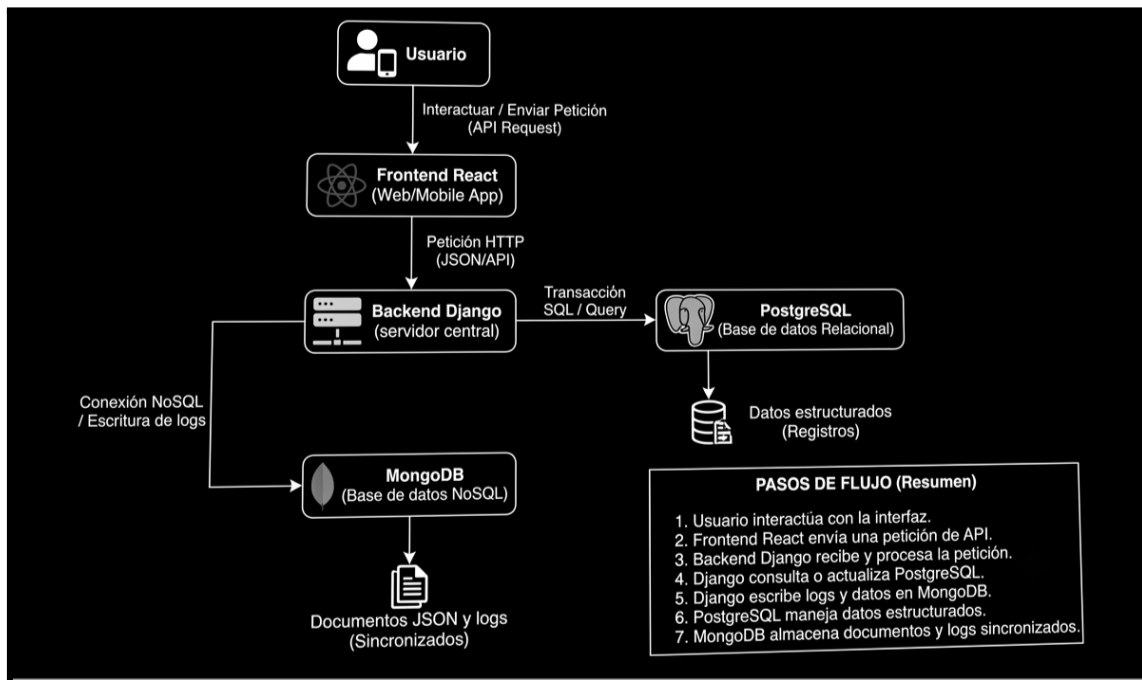
5.1.5. Bases de datos (MongoDB): Parte complementaria del sistema que permite guardar información de forma más flexible mediante documentos JSON, y almacena datos adicionales obtenidos desde PostgreSQL, y registra los procesos de sincronización realizados, esta hace uso de colecciones como “pokemon_details”, donde se guardan detalles de Pokémon, y “sync_logs”, donde se almacenan los registros de las sincronizaciones.

5.2. Diagrama de componentes.

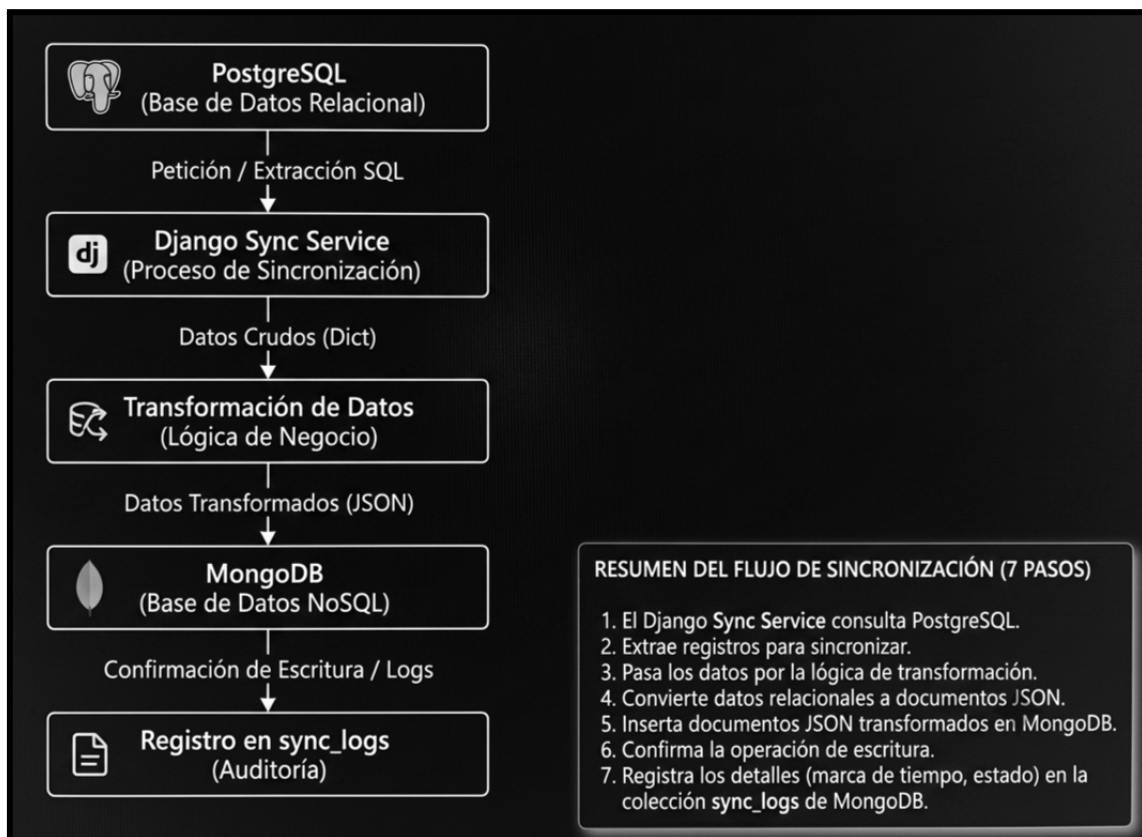


5.3. Flujo del sistema.

5.3.1. Flujo general del sistema.



5.3.2. Flujo de la sincronización.



5.4. Modelo de datos.

El modelo de datos se realizó en base a consideración de los requisitos y las características de cada base de datos dentro de una arquitectura distribuida, y la combinación de los dos modelos permite aprovechar las ventajas de cada tecnología, por un lado proporcionando estructura, integridad, y la consistencia de la información, y por otro lado permitiendo la información flexible que se deriva del proceso de transformación y sincronización

5.4.1. Modelo relacional PostgreSQL.

Esta base de datos representa el motor principal de almacenamiento relacional, se encarga de administrar la información estructurada de pokémon, de entrenadores y de usuarios, la integración con el mapeo objeto - relacional (ORM) de django permite transformar las entidades del sistema en tablas con campos definidos para cada atributo, asimismo se incorporan estructuras propias del framework para gestionar autenticación, sesiones, y permisos, garantizando un manejo seguro de accesos, y una correcta organización de los datos utilizados por la aplicación.

5.4.2. Modelo documental MongoDB.

Esta base de datos permite la implementación de un modelo documental basado en JSON que permite gestionar información flexible dentro de la arquitectura, y con la colección “pokemon_details” almacena documentos generados desde PostgreSQL mediante procesos de transformación, incorporando atributos adicionales, y la colección “sync_logs” que se encarga de registrar los eventos de sincronización entre bases de datos, con campos como la fecha, el origen, el destino, la cantidad de registros, y el estado del proceso, facilitando el seguimiento de operaciones distribuidas.

5.4.3. Cuadro comparativo de modelo de datos.

MODELO DE DATOS		
CARACTERÍSTICAS	POSTGRESQL	MONGODB
Modelo de almacenamiento	Relacional	Documental
Estructura	Tablas con filas y columnas	Documentos JSON
Esquema	Estructurado y definido	Flexible
Datos principales	Pokémon, entrenadores y usuarios	Información documental y logs
Relaciones	Mediante claves y	Mediante documentos

	restricciones	independientes
Integridad	Controlada por reglas relacionales	Controlada por la aplicación
Escalabilidad	Vertical y optimización relacional	Flexible para grandes volúmenes documentales
Uso dentro del proyecto	Base principal	Base complementaria

5.5. Reglas de transformación y sincronización.

La integración de las bases de datos debe ser realizada mediante los procesos de transformación de datos que convierten registros relacionales en documentos JSON compatibles, por ello este mecanismo debe aplicar reglas que son establecidas para preservar la coherencia, la estructura, y la integridad de la información durante la sincronización entre ambas BD.

5.5.1. Transformación postgresQL a mongoDB.

La transformación de los datos desde postgresQL hacia mongoDB se basa en la obtención de los registros que se almacenan en la tabla de pokémon y logra convertir cada registro relacional en un documento JSON, el debido proceso que se sigue para esta transformación es:

- Se obtienen todos los registros de pokémon almacenados en la base de datos principal (PostgreSQL) mediante el ORM.
- Extrae los valores debidos de cada atributo del modelo relacional.
- Construye un documento JSON con una estructura adaptada al modelo documental de mongoDB.
- Agrega cada documento dentro de la colección “pokemon_details”.
- Registra la operación realizada dentro de la colección “sync_logs”.

5.5.2. Transformación mongoDB a postgresQL.

La transformación de los datos desde mongoDB hacia postgresQL se basa en convertir los documentos almacenados en mongoDB a registros compatibles con postgresQL, logrando la comunicación bidireccional, el proceso que se sigue para esta transformación es:

- Obtener documentos almacenados en mongoDB.
- Validar que los campos estén disponibles.
- Cambiar todos los atributos documentales al formato utilizado por postgresQL.

- Verificar las restricciones de integridad antes de insertar o actualizar los registros.
- Registrar el resultado del proceso dentro del sistema de logs.

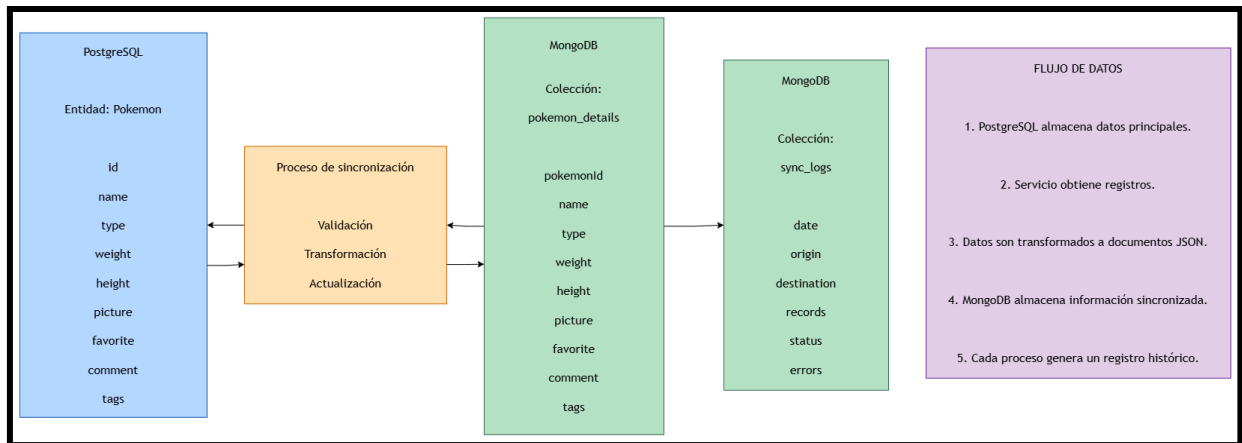
5.5.3. Reglas de mapeo de datos

El proceso de transformación aplica reglas de correspondencia entre los campos de postgresSQL y las estructuras documentales de mongoDB, deben realizarse las conversiones directas, las adaptaciones de rutas, y la generación de valores predeterminados, garantizando una migración organizada y consistente de la información sincronizada. De esta forma con el mapeo campo a campo se puede establecer una relación entre los atributos de postgresSQL y los atributos de mongoDB, para que durante la sincronización se respeten los significados, el tipo de dato, y se mantenga una consistencia de la información entre las bases de datos.

- **Tabla de relaciones.**

RELACIONES ENTRE BD		
POSTGRESQL	MONGODB	DESCRIPCIÓN
id	pokemonId	Identificador del Pokémon
name	name	Nombre
type	type	Tipo
weight	weight	Peso
height	height	Altura
picture	picture	Ruta de la imagen
favorite	favorite	Pokémon favorito
comment	comment	Comentario
tags	tags	Etiquetas

- **Diagrama de relaciones.**



Sincronización bidireccional de datos.

Al obtener la sincronización bidireccional se permite el manejo de la información entre postgresQL, y mongoDB mediante procesos de actualización en ambos sentidos, es decir enviar datos de mongo a postgres y viceversa, esto requiere validación documental, transformación de estructuras JSON, aplicación de restricciones relacionales y actualización de registros, garantizando la consistencia, y reduciendo la dependencia de una única fuente de datos.

5.5.4. Registro de sincronización.

Con la implementación de “sync_logs” que se añade en mongoDB se puede registrar, y gestionar las ejecuciones de los procesos de sincronización dentro de la arquitectura, cada documento almacena datos relevantes como la fecha de ejecución, el origen, el destino, la cantidad de registros procesados, y el estado final de la operación, con este mecanismo se proporciona un historial de actividades que facilita el monitoreo, control y análisis de los intercambios de información entre las bases de datos.

6. Implementación del sistema.

6.1. Tecnologías utilizadas.

6.1.1. Python: Con esta tecnología se logró crear el sistema esto debido a la facilidad para entender, para modificar y por el acceso a las herramientas, y con esto se realizó el manejo de los datos y la conexión a las bases de datos.

6.1.2. Django: Con esta tecnología se pudo desarrollar la página web debido a que nos permite organizar el código por su estructura basada en el patrón MVC, y facilita la creación de las funciones del sistema, asimismo es más sencillo manejar formularios, usuarios, archivos, y conectar con la base de datos.

- 6.1.3. PostgreSQL:** Esta tecnología fue implementada como la base de datos relacional principal del sistema que sirve para almacenar las entidades, tiene cualidades como su robustez y su soporte para su integridad asegurando que exista consistencia, organización, y confiabilidad de la información guardada.
- 6.1.4. MongoDB:** Esta tecnología fue implementada como base de datos documental para almacenar información sincronizada de los pokemones y el historial de procesos, haciendo que la estructura sea flexible y permitió agregar nuevos campos sin modificar el modelo relacional, de esta manera se pudo facilitar el manejo de la información adicional del sistema.
- 6.1.5. PyMongo:** Esta librería nos sirvió para conectar django con mongoDB, para lograr intercambiar la información, con esta librería se volvió posible guardar, consultar, modificar, y eliminar datos de los pokemones, y así controlar los registros de sincronización entre las dos bases de datos.
- 6.1.6. Django rest framework:** Esta tecnología sirvió para la construcción de la API REST, con esta herramienta crear endpoints para la consulta y la administración de la información, serializar objetos e incluso el intercambio de datos en formato JSON fue más sencillo.

6.2. Configuración del entorno.

Con el desarrollo del sistema se deben realizar una serie de configuraciones del entorno de trabajo, para integrar los servicios de django, de las bases de datos (PostgreSQL y mongoDB), y lograr establecer la comunicación entre el sistema y las bases de datos, además de las librerías. Para esto se fue configurar el entorno fue necesario realizar los siguientes pasos:

- Instalación del entorno de desarrollo.
- Configuración del proyecto django.
- Configuración de postgresQL.
- Configuración de mongoDB.
- Ejecución del sistema.

Para comprender de manera más detallada las configuraciones observar el anexo 1.

6.3. Implementación de la API Rest.

La api se implementó bajo el funcionamiento del modelo cliente - servidor, de esta forma se reciben solicitudes HTTP, las cuales son procesadas por django, y las respuestas se envían en formato JSON, para lograr desarrollar esto se configura django rest framework (Para instalar se utiliza el comando “pip install djangorestframework” y se agrega al archivo de configuración del proyecto) que tiene la capacidad de incorporar serializadores (Sirve para transformar los objetos del modelo en datos JSON), manejo de solicitudes, y así se logra la comunicación eficiente entre el cliente, postgresQL, y mongoDB, asimismo se agregaron endpoints en la api para manejar los recursos relacionados con los pokemones mediante diferentes métodos HTTP (Get, post, put, patch, delete) y cada endpoint emplea rutas definidas y proporciona respuestas en formato JSON para facilitar la comunicación entre el cliente y el servidor, y finalmente la api además de trabajar con postgres, fue preparada para trabajar con la información guardada en mongo logrando también consultar la información de las colecciones agregadas.

6.4. Descripción de la implementación del código.

6.4.1. Implementación del modelo de datos de pokemones en PostgreSQL.

Se implementó el modelo pokemon con el fin de representar la estructura de almacenamiento principal en postgresQL, este modelo se creó con los campos necesarios para gestionar características como el nombre, el tipo, el peso, la altura y la imagen, además de atributos complementarios como favorito, comentarios y etiquetas, con estos nuevos campos se puede ampliar la información registrada y se facilitan los procesos de sincronización con mongoDB. (Código visible en el anexo 2 - 10.2.1).

6.4.2. Configuración e integración de mongoDB.

Para esta integración se modificó el archivo “services/mongo.py” donde se conecta django con mongoDB mediante la librería pymongo, asimismo se utilizaron los archivos “services/sync_service.py” y “pokedex/views.py” para integrar sus funciones, este cambio fue realizado para agregar la segunda base de datos con la capacidad de guardar información flexible de los pokemones y los registros de sincronización, con esta conexión se puede crear, consultar y actualizar documentos en las colecciones “pokemon_details” y “sync_logs”. (Código visible en el anexo 2 - 10.2.2).

6.4.3. Creación del sistema de sincronización postgresQL a mongoDB.

Se implementó en el archivo “services/sync_service.py” la función “sync_postgres_to_mongo()”, la cual es encargada de realizar la sincronización de datos desde PostgreSQL hacia MongoDB, con esta modificación es necesario establecer un mecanismo de comunicación entre ambas bases de datos, donde el proceso obtiene los registros mediante django ORM, hace valida la información, corrige las inconsistencias, transforma los datos al formato documental y los almacena en la colección “pokemon_details” y finalmente, genera un registro en “sync_logs” para un control y seguimiento de las operaciones ejecutadas. (Código visible en el anexo 2 - 10.2.3).

6.4.4. Creación del sistema de sincronización mongoDB a postgresQL.

Se modificó el archivo “services/sync_service.py” para crear la función “sync_mongo_to_postgres()”, la cual envía información desde mongoDB hacia postgresQL, con este cambio fue necesario mantener actualizadas ambas bases de datos y permitir una sincronización en ambos sentidos, por ello la función debe obtener los documentos de “pokemon_details”, se deben revisar los datos, corregir errores, y finalmente convertir la información al formato de postgresQL, actualizando los registros en el modelo pokemon, asimismo guarda un registro en “sync_logs” para controlar las operaciones realizadas. (Código visible en el anexo 2 - 10.2.4).

6.4.5. Implementación del historial de sincronizaciones.

Se modificaron los archivos “services/sync_service.py”, “pokedex/views.py”, “pokedex/urls.py” y “templates/sync_logs.html”, además se creó “sync_logs” en mongoDB, el cambio se basó en agregar un historial de la sincronización realizada entre postgresQL a mongoDB, y viceversa, esto fue implementado porque era necesario conocer cuándo se ejecutaban los procesos, la cantidad de registros que se transferían, los errores que ocurrían, y básicamente su funcionamiento se basa en guardar la información después de cada sincronización, consultar los registros almacenados y mostrarlos en una interfaz. (Código visible en el anexo 2 - 10.2.5).

6.4.6. Implementación de operaciones CRUD en mongoDB.

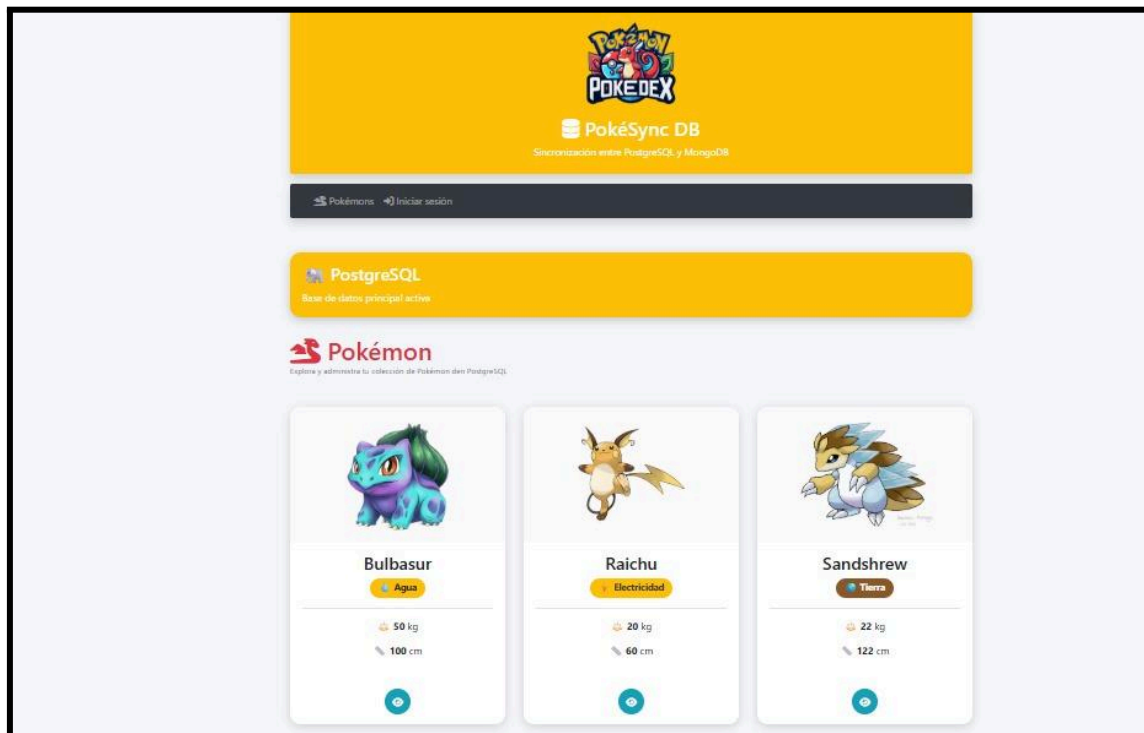
Se implementaron operaciones CRUD para la base de datos mongoDB haciendo modificaciones en “pokedex/views.py”, “pokedex/urls.py” y los templates “mongo_index.html”, “mongo_form.html” y “mongo_detail.html”, y asimismo haciendo uso de la colección “pokemon_details”, esta modificación busca que se logre

crear, consultar, modificar y eliminar documentos almacenados en la base documental es decir mongoDB, y así se otorga una independencia a mongoDB y no solo es una base de datos de sincronización esto permitirá administrar datos y validar procesos de sincronización.

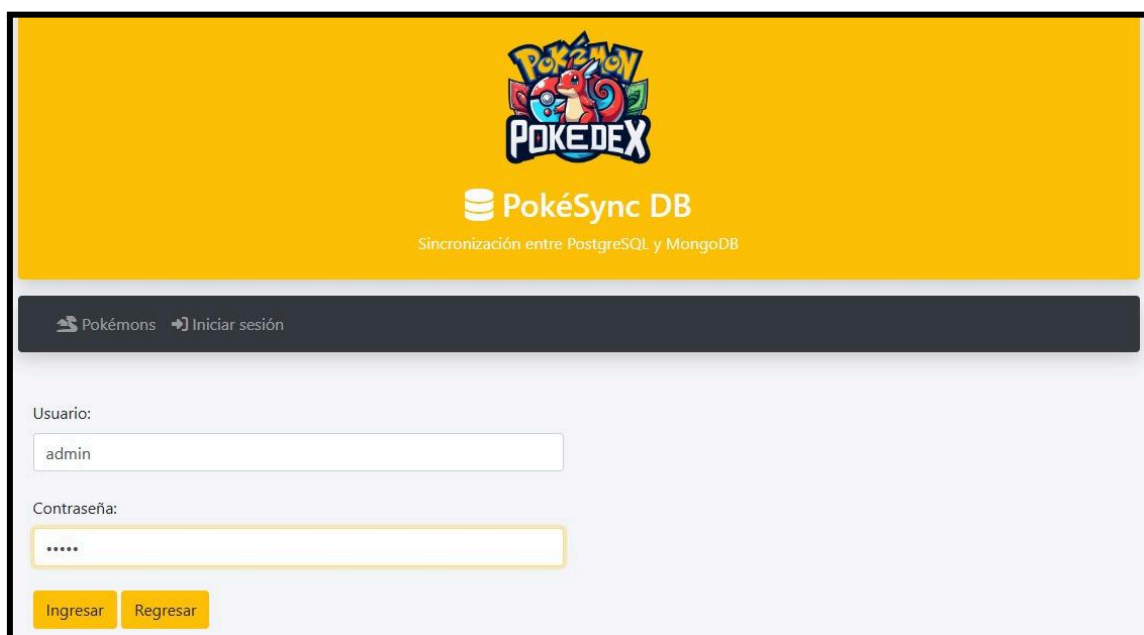
6.5. Pruebas del sistema.

6.5.1. Pruebas funcionales.

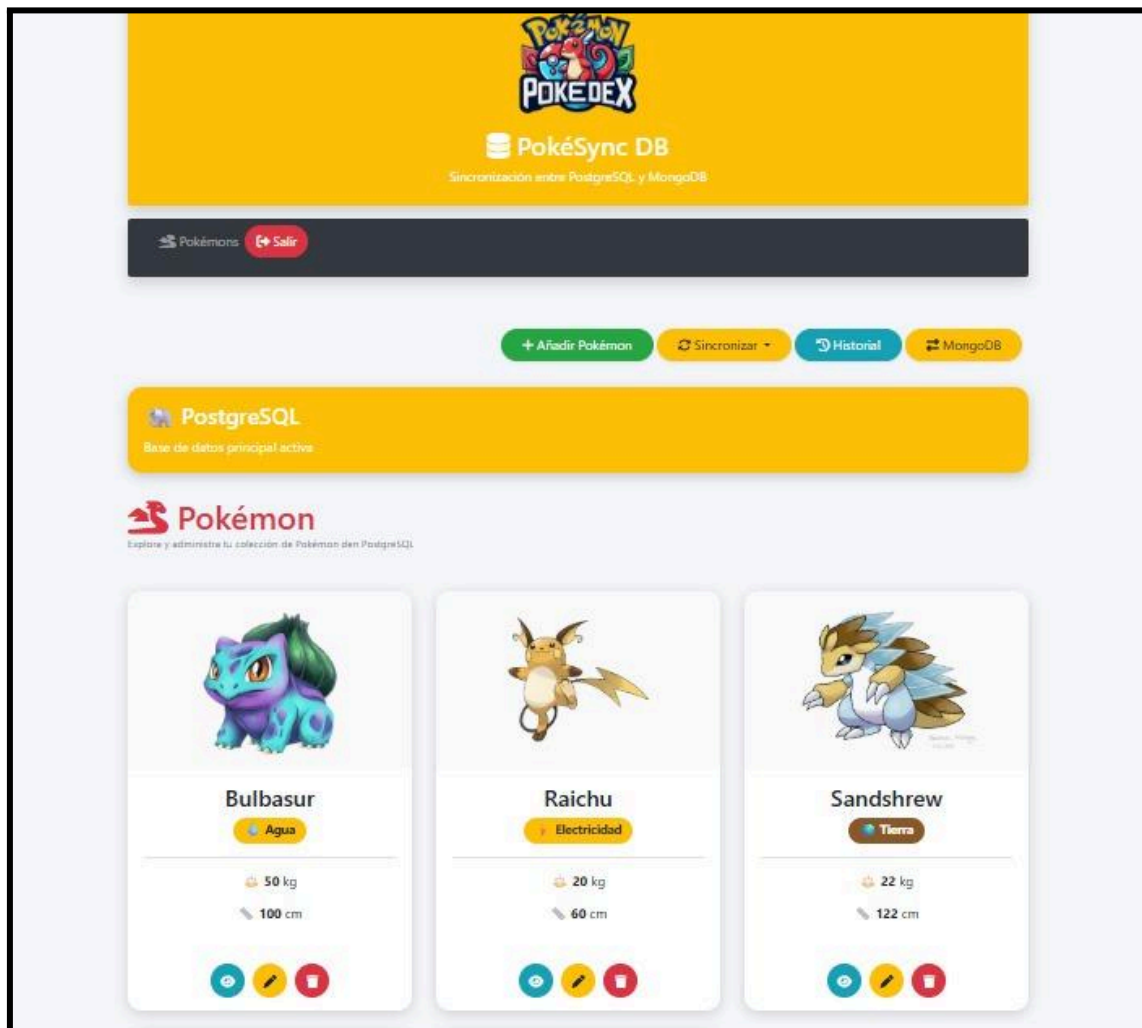
- Interfaz sin inicio de sesión.



- Login.



- Iniciando sesión (PostgreSQL).



- Añadir pokemon.



PokéSync DB
Sincronización entre PostgreSQL y MongoDB

Pokémons
Safe

Agregar Pokémon (PostgreSQL)

Nombre

Tipo

Peso

Altura

Imagen

Sin archivos seleccionados

☐ Pokémon favorito

Comentario

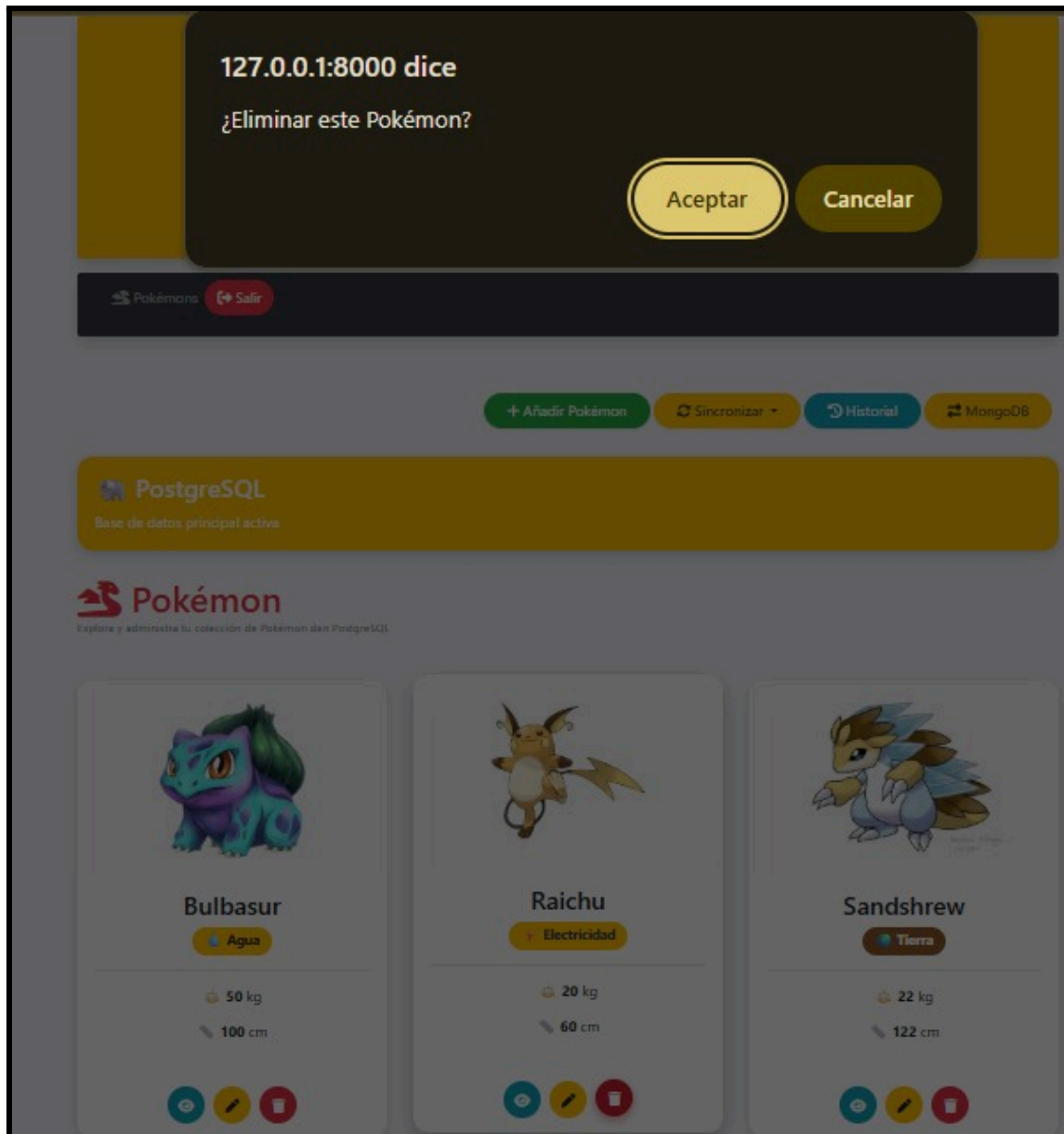
Etiquetas

Ej: Eléctrico, Inicial, Raro

Separe las etiquetas usando comas. Separe las etiquetas usando comas...

Guardar Pokémon
Cancelar

- **Eliminar pokemon.**



- Editar pokemon.

Editar Pokémon (PostgreSQL)

Nombre
Bulbasur

Tipo
Agua

Peso
50

Altura
100



Imagen Currently: [pokemon_pictures/temp_nQZT&Za.png](#) ☐ Clear

Change:
 Sin archivos seleccionados

☐ Pokémon favorito

Comentario

Etiquetas

Separe las etiquetas usando comas. Separe las etiquetas usando comas.

- Botón de sincronización.



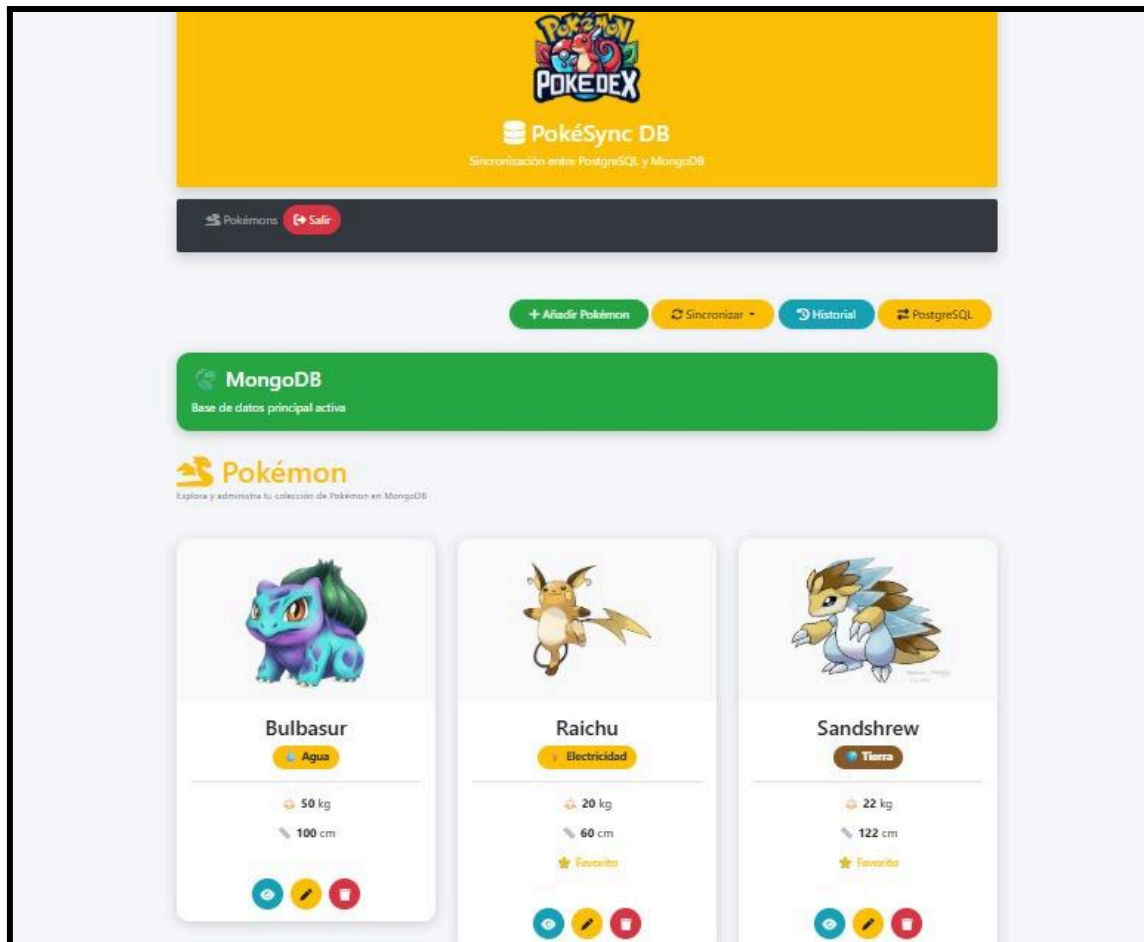
PokéSync DB
Sincronización entre PostgreSQL y MongoDB





PostgreSQL
Base de datos principal activa

- Iniciando sesión (MongoDB).



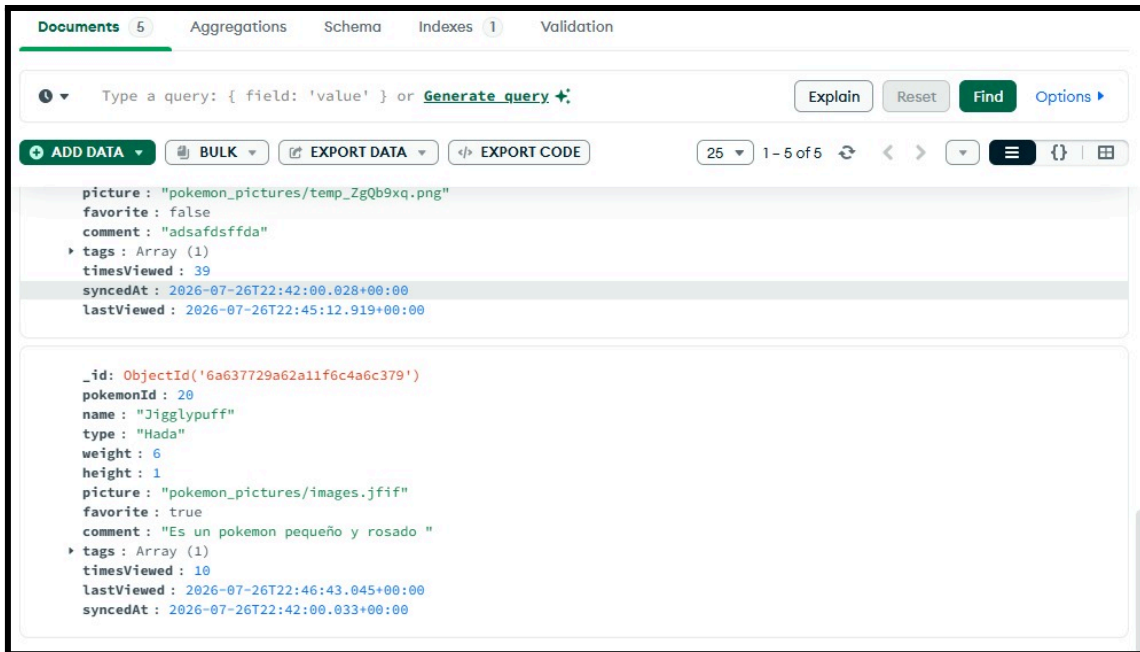
- Pre - sincronización (PostgreSQL).

Data Output Messages Notifications									
Showing rows: 1 to 5 Page No: 1 of 1									
	id	name	type	weight	height	picture	comment	favorite	tags
	bigint	character varying (100)	character varying (140)	integer	integer	character varying (100)	text	boolean	text
1	1	Bulbasur	Agua	50	100	pokemon_pictures/temp_nQT8Za.png		false	
2	2	Raichu	Electricidad	20	60	pokemon_pictures/026.png	Este es el mejor pokemon	true	
3	13	Sandshrew	Tierra	22	122	pokemon_pictures/temp_EE72E3w.we...	Este parece un puercoespín	true	Inicial, Kan...
4	18	Steelix	Piedra	50	59	pokemon_pictures/temp_ZgQb9xq.png	adsafdsffda	false	Raro
5	20	Jigglypuff	Hada	6	1	pokemon_pictures/images.jfif	Es un pokemon pequeño y rosa...	true	Planta

- Post - sincronización (PostgreSQL).

Data Output Messages Notifications									
Showing rows: 1 to 6 Page No: 1 of 1									
	id	name	type	weight	height	picture	comment	favorite	tags
	[PK] bigint	character varying (100)	character varying (140)	integer	integer	character varying (100)	text	boolean	text
1	1	Bulbasur	Agua	50	100	pokemon_pictures/temp_nQT8Za.p...		false	
2	2	Raichu	Electricidad	20	60	pokemon_pictures/026.png	Este es el mejor po...	true	
3	13	Sandshrew	Tierra	22	122	pokemon_pictures/temp_EE72E3w.w...	Este parece un pue...	true	Kanto
4	18	Steelix	Piedra	50	59	pokemon_pictures/temp_ZgQb9xq.p...	adsafdsffda	false	Raro
5	20	Jigglypuff	Hada	6	1	pokemon_pictures/images.jfif	Es un pokemon pe...	true	Plan...
6	28	Aurorus	Roca	32	45	pokemon_pictures/images_6vnYk4e....	Fue regenerado a p...	true	Raro

- Pre - sincronización (MongoDB).

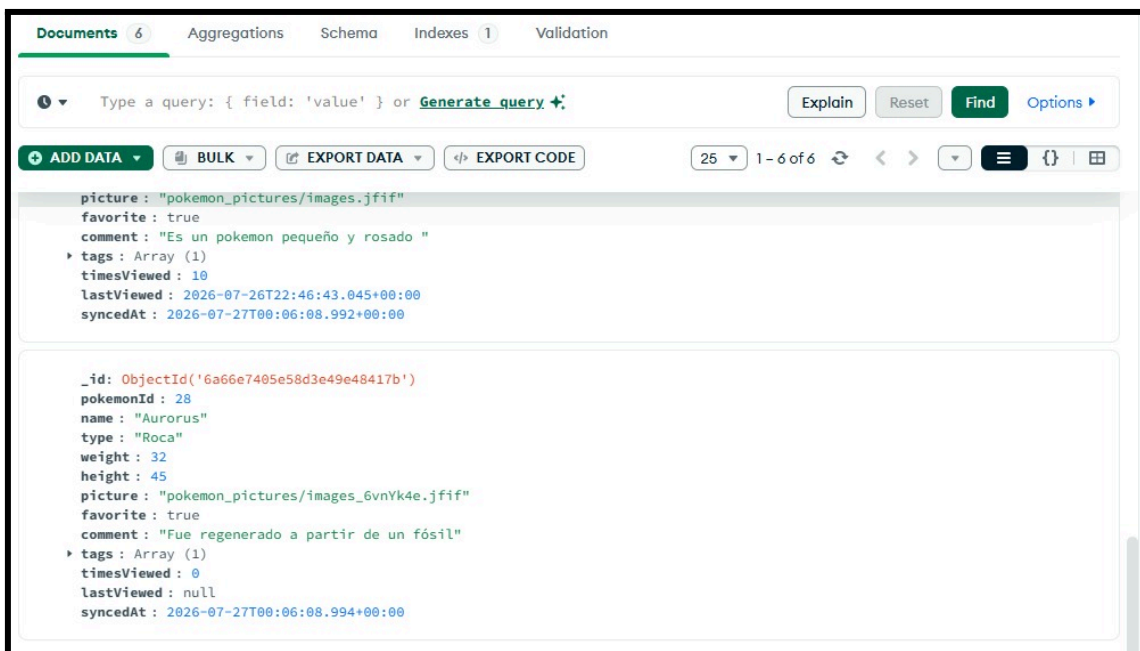


The screenshot shows the MongoDB Compass interface. The 'Documents' tab is active, displaying a list of documents. The selected document is for a Jigglypuff, with the following details:

```

_id: ObjectId('6a637729a62a11f6c4a6c379')
pokemonId: 20
name: "Jigglypuff"
type: "Hada"
weight: 6
height: 1
picture: "pokemon_pictures/images.jfif"
favorite: true
comment: "Es un pokemon pequeño y rosado "
tags: Array (1)
timesViewed: 10
lastViewed: 2026-07-26T22:46:43.045+00:00
syncedAt: 2026-07-26T22:42:00.033+00:00
  
```

- **Post - sincronización (MongoDB).**



The screenshot shows the MongoDB Compass interface. The 'Documents' tab is active, displaying a list of documents. The selected document is for an Aurorus, with the following details:

```

_id: ObjectId('6a66e7405e58d3e49e48417b')
pokemonId: 28
name: "Aurorus"
type: "Roca"
weight: 32
height: 45
picture: "pokemon_pictures/images_6vnYk4e.jfif"
favorite: true
comment: "Fue regenerado a partir de un fósil"
tags: Array (1)
timesViewed: 0
lastViewed: null
syncedAt: 2026-07-27T00:06:08.994+00:00
  
```

6.6. Resultados esperados y obtenidos.

RESULTADOS ESPERADOS VS OBTENIDOS		
ASPECTO	R. ESPERADO	R. OBTENIDO
Integración de postgresSQL y mongoDB	Implementar una arquitectura híbrida que integre postgresSQL y mongoDB, permitiendo	Se integraron PostgreSQL y MongoDB mediante Django, utilizando PostgreSQL como base

	gestionar información desde ambas bases de datos dentro del mismo sistema.	principal y MongoDB para almacenar datos flexibles e historiales de sincronización.
Modelo de datos de Pokémon en PostgreSQL	Se creó una estructura para almacenar datos básicos de pokemones, permitiendo incorporar nuevas características y ampliaciones futuras del sistema.	Se implementó el modelo pokemon en PostgreSQL con campos adicionales, permitiendo almacenar información detallada como favoritos, comentarios y etiquetas.
Sincronización PostgreSQL a MongoDB	Permitir transferir información almacenada en PostgreSQL hacia MongoDB.	Se implementó la función <code>sync_postgres_to_mongo()</code> , la cual obtiene registros mediante django ORM.
Sincronización MongoDB a PostgreSQL	Permitir enviar modificaciones realizadas en MongoDB hacia PostgreSQL	Se desarrolló la función <code>sync_mongo_to_postgres()</code> , capaz de leer documentos desde MongoDB.
Historial de sincronizaciones	Registrar información sobre cada proceso.	Se creó la colección <code>sync_logs</code> en MongoDB, donde se guarde con fecha, origen, destino, cantidad de registros procesados, errores encontrados y errores solucionados.
Visualización del historial de procesos	Mostrar al usuario información sobre las sincronizaciones realizadas mediante una interfaz gráfica.	Se implementó una sección donde se puede ver el historial de sincronizaciones almacenados en <code>sync_logs</code> , permitiendo consultar el

		historial de operaciones realizadas.
Operaciones CRUD en mongoDB	Permitir administrar los documentos almacenados en mongoDB sin depender exclusivamente del proceso de sincronización.	Se implementaron operaciones para crear, consultar, actualizar y eliminar sobre la colección pokemon_details, para así manejar los pokémones directamente desde mongoDB.
Interfaz gráfica del sistema	Mejorar la interacción del usuario mediante una interfaz organizada y accesible.	Se actualizó la interfaz, agregando botones, formularios, tarjetas de pokémon y lograr administrar la información desde postgresQL o mongoDB.

6.7. Limitaciones y trabajo futuro.

LIMITACIONES Y TRABAJO FUTURO	
LIMITACIÓN	FUTURO
La conexión entre las dos bases de datos debe iniciarse manualmente por el usuario desde la pantalla del sistema.	Crear un sistema que realice sincronizaciones automáticamente en horarios definidos, sin que el usuario tenga que iniciarlas manualmente.
El sistema no decide automáticamente qué cambió conservar cuando un mismo dato es modificado al mismo tiempo en ambas bases.	Agregar reglas para elegir qué cambio conservar cuando existen modificaciones diferentes, usando fechas o versiones de los datos.
El sistema fue probado con pocos datos, por lo que puede necesitar más pruebas antes de manejar grandes cantidades de	Optimizar la sincronización para manejar grandes volúmenes de información mediante procesos por lotes e índices.

información.	
El historial guarda datos básicos sobre las sincronizaciones realizadas, como fechas, registros enviados y resultados obtenidos durante cada proceso.	Ampliar el módulo de reportes incorporando estadísticas, filtros y análisis de rendimiento para evaluar los procesos de sincronización.

7. Conclusiones.

- La implementación de la pokédex permitió comprender que integrar diferentes modelos de bases de datos permite aprovechar las ventajas de cada tecnología. y hacer que se complementen entre ellas, por un lado mejorando puntos como la organización, y la consistencia para datos estructurados, y por otro dando flexibilidad para información complementaria, dando como resultado una arquitectura híbrida que refleja cómo los sistemas actuales pueden combinar soluciones relacionales y NoSQL para mejorar adaptación, escalabilidad y gestión de información.
- La transformación y sincronización de datos evidenciaron la importancia de establecer procesos controlados entre diferentes bases de datos, ya que la conversión entre modelos relacional y documental requiere ciertas validaciones, reglas definidas, y registros de operaciones que logren garantizar la consistencia, la trazabilidad, y el correcto intercambio de información dentro de sistemas distribuidos.
- La implementación del sistema permitió evidenciar que el uso de una arquitectura basada en diferentes motores de bases de datos puede atender de manera eficiente las necesidades específicas de almacenamiento y consulta y al asignar las responsabilidades entre postgresSQL y mongoDB favorece una organización adecuada de la información según su naturaleza, demostrando que la integración de diversas tecnologías constituye una alternativa funcional para desarrollar soluciones.

8. Recomendaciones.

- Se recomienda agregar herramientas de análisis y visualización de datos para ampliar las capacidades de la aplicación, ya sea un módulo con estadísticas, gráficos y paneles que permitan la interpretación de los datos, con la capacidad de identificar tendencias, o lograr una pokedex analítica aprovechando la versatilidad del sistema.

- Se recomienda mejorar la arquitectura del sistema para permitir un manejo eficiente de mayores cantidades de información y usuarios, para lograrlo, se pueden implementar índices en PostgreSQL y MongoDB, sistemas de caché y técnicas de paginación.
- Se recomienda implementar procedimientos periódicos de respaldo para PostgreSQL y MongoDB, considerando las diferencias existentes en sus estructuras de almacenamiento, el tener respaldos actualizados ayuda a proteger los datos ante eliminaciones accidentales, errores de los usuarios o fallas técnicas, así si ocurre algún problema, la información puede recuperarse con mayor facilidad y seguridad.

9. Bibliografía.

- Bienvenido a MongoDB Docs - Documentación de MongoDB - MongoDB Docs. (s. f.). MongoDB Documentation. https://www.mongodb.com/es/docs/?utm_source=chatgpt.com
- PostgreSQL: documentation. (s. f.). The PostgreSQL Global Development Group. https://www.postgresql.org/docs/?utm_source=chatgpt.com
- Wikipedia contributors. (2024, 6 agosto). Heterogeneous database system. Wikipedia. https://en.wikipedia.org/wiki/Heterogeneous_database_system
- O'Connor, Q. (2023, 5 septiembre). NoSQL vs. Relational Databases. DB Services. <https://dbservices.com/blog/nosql-vs-relational-database>
- Admin. (2026, 6 mayo). Sincronización de datos en bases de datos SQL. ABC Data Soluciones. <https://abcdatasoluciones.com/sincronizacion-de-datos-en-bases-de-datos-sql/>

10. Anexos.

10.1. Anexo 1: Configurar el entorno.

- **Instalación del entorno de desarrollo.**

Para el desarrollo del sistema se debe preparar un ambiente que sea aislado con la finalidad de gestionar bien las dependencias del proyecto, por ello se preparó un entorno virtual python para evitar conflictos entre versiones de paquetes y facilitar la instalación del proyecto en otros equipos, la creación del entorno se debe realizar mediante el comando “python -m venv venv” y debe ser activado con el comando “venv\Scripts\activate”. Una vez sea activado el entorno se deben instalar las dependencias necesarias para el correcto funcionamiento del sistema con el comando “pip install -r requirements.txt” ya que aquí tenemos las dependencias instaladas por el

desarrollador algunas de estas dependencias son: django, django REST Framework, psycopg2, y pyMongo.

- **Configuración del proyecto django.**

Una vez sea configurado el entorno python, se debe implementar MVC, para separar las responsabilidades mediante modelos que representen las entidades almacenadas, definiendo principalmente el modelo de pokémon, incluyendo campos como características, imágenes, estados, comentarios y datos personales asociados a cada registro.

- **Configuración de postgresQL.**

Para configurar la conexión con postgresQL, se logra con el ORM que es propio de django, así se interactúa los modelos python con la base de datos, para configurar esto debemos entrar al archivo “Settings.py” específicamente en el apartado de “Databases” donde se deben definir los parámetros como el nombre de la base, el usuario, la contraseña, el host, y el puerto, con la finalidad de establecer la comunicación con el servidor PostgreSQL, una vez concretada la conexión, se deben generar las tablas correspondientes en base a los modelos creados, eso lo logramos con el comando “python manage.py makemigrations”, y luego con el comando “python manage.py migrate” para aplicar los cambios a la base de datos, logrando así que django cree automáticamente las estructuras necesarias para almacenar la información del sistema.

- **Configuración de mongoDB.**

Para configurar la conexión con mongoDB como segunda fuente de almacenamiento se debe establecer la comunicación entre django y mongo, mediante el uso de pymongo, y así lograr insertar documentos, consultar información, actualizar registros, eliminar documentos y registrar procesos de sincronización, para esto también se creó dos colecciones, la primera es “pokemon_details” la cual sirve para guardar la información documental de los pokémones y también estableciendo los campos principales que corresponden al mapeo realizado desde postgresQL hacia mongoDB, y la segunda es “sync_logs” la cual sirve para guardar el historial de sincronizaciones registrando datos como la fecha de ejecución, base de datos origen y destino, procesos realizados, estado, errores, y soluciones.

- **Ejecución del sistema.**

Una vez configurados los componentes del sistema, la aplicación fue iniciada mediante el servidor interno de django utilizando el comando “python manage.py runserver”,

logrando ejecutar el entorno local y acceder a las funcionalidades mediante un navegador web.

10.2. Anexo 2: Códigos relevantes.

10.2.1. Implementación del modelo de datos de pokemones en PostgreSQL.

```
from django.db import models

class Pokemon(models.Model):
    name = models.CharField(max_length=100, verbose_name="Nombre")
    type = models.CharField(max_length=100, verbose_name="Tipo")
    weight = models.IntegerField(verbose_name="Peso")
    height = models.IntegerField(verbose_name="Altura")
    picture = models.ImageField(upload_to="pokemon_pictures/", blank=True,
    null=True, verbose_name="Imagen")
    favorite = models.BooleanField(default=False, verbose_name="Favorito")
    comment = models.TextField(blank=True, default="", verbose_name="Comentario")
    tags = models.CharField(max_length=255, blank=True, default="",
    verbose_name="Etiquetas")
    def __str__(self):
        return self.name
```

10.2.2. Configuración e integración de mongoDB.

- **services/mongo.py**

```
from pymongo import MongoClient
client = MongoClient( "mongodb://localhost:27017/")
db = client["pokedex"]
pokemon_details = db["pokemon_details"]
sync_logs = db["sync_logs"]
```

Se modificó para conectar django con mongoDB, agregando las colecciones para pokemon e historial, permitiendo guardar, consultar y sincronizar información.

- **services/sync_service.py**

```
from pokedex.models import Pokemon
from services.mongo import (pokemon_details, sync_logs)
from datetime import datetime
```

Se modificó para conectar postgresQL y mongoDB, donde se agregaron importaciones necesarias para sincronizar datos, colecciones y registrar fechas.

- **pokedex/views.py**

```
from services.mongo import (pokemon_details, sync_logs)
```

Se modificó para conectar las vistas con mongoDB, y esto permitió mostrar, crear, editar, eliminar pokemon y consultar historiales.

- **pokedex/urls.py**

```
from . import views
```

```
urlpatterns = [
```

```
    path("mongo/", views.mongo_index, name="mongo_index"),
```

```
        path("mongo/edit/<int:id>/", views.edit_mongo_pokemon,
```

```
        name="edit_mongo_pokemon"),
```

```
        path("mongo/delete/<int:id>/", views.delete_mongo_pokemon,
```

```
        name="delete_mongo_pokemon"),
```

```
]
```

Se modificó para crear rutas de mongoDB, y con estas se pueda visualizar, editar y eliminar pokemones mediante la interfaz web.

10.2.3. Creación del sistema de sincronización postgresQL a mongoDB.

```
from pokedex.models import Pokemon
```

```
from services.mongo import pokemon_details, sync_logs
```

```
from datetime import datetime
```

```
def sync_postgres_to_mongo():
```

```
    cantidad = 0
```

```
    ids_postgres = [
```

```
        pokemon.id
```

```
        for pokemon in Pokemon.objects.all()
```

```
]
```

```
    pokemon_details.delete_many({
```

```
        "pokemonId": {
```

```
            "$nin": ids_postgres
```

```
        }
```

```
    })
```

```
    for pokemon in Pokemon.objects.all():
```

```
        documento_existente = pokemon_details.find_one(
```

```
            {"pokemonId": pokemon.id})
```

```

)
if documento_existente:
    pokemon_details.update_one(
        {"pokemonId": pokemon.id},
        {
            "$set": {
                "name": pokemon.name,
                "type": pokemon.type,
                "weight": pokemon.weight,
                "height": pokemon.height,
                "picture": str(pokemon.picture),
                "syncedAt": datetime.now()
            }
        }
    )
else:
    pokemon_details.insert_one({
        "pokemonId": pokemon.id,
        "name": pokemon.name,
        "type": pokemon.type,
        "weight": pokemon.weight,
        "height": pokemon.height,
        "picture": str(pokemon.picture),
        "favorite": pokemon.favorite,
        "comment": pokemon.comment,
        "tags": (
            pokemon.tags.split(",")
            if pokemon.tags else []
        ),
        "timesViewed": 0,
        "lastViewed": None,
        "syncedAt": datetime.now()
    })
cantidad += 1

```

```

sync_logs.insert_one({
    "date": datetime.now(),
    "origin": "PostgreSQL",
    "destination": "MongoDB",
    "records": cantidad,
    "status": "SUCCESS"
})
return cantidad

```

Se implementó para transferir datos entre bases, el proceso válida, actualiza documentos y registra sincronizaciones.

10.2.4. Creación del sistema de sincronización mongoDB a postgresQL.

```

def sync_mongo_to_postgres():
    documentos = pokemon_details.find()
    cantidad = 0
    for doc in documentos:
        try:
            pokemon = Pokemon.objects.get(id=doc["pokemonId"])
            pokemon.favorite = doc.get("favorite", False)
            pokemon.comment = doc.get("comment", "")
            pokemon.tags = ", ".join(
                doc.get("tags", [])
            )
            pokemon.save()
            cantidad += 1
        except Pokemon.DoesNotExist:
            pass
    sync_logs.insert_one({
        "date": datetime.now(),
        "origin": "MongoDB",
        "destination": "PostgreSQL",
        "records": cantidad,
        "status": "SUCCESS"
    })

```



```

    return cantidad
def get_sync_logs():
    logs = list(
        sync_logs.find().sort("date", -1)
    )
    return logs

```

Se modificó para sincronizar mongoDB con postgresQL, aquí la función convierte, valida y guarda datos, manteniendo ambas bases conectadas.

10.2.5. Implementación del historial de sincronizaciones.

- **services/sync_service.py**

```

sync_logs.insert_one(
    {
        "date": datetime.now(),
        "origin": "PostgreSQL",
        "destination": "MongoDB",
        "records": cantidad,
        "status": "SUCCESS",
        "errorsFound": errores_encontrados,
        "errorsSolved": errores_resueltos
    }
)

```

Se agregó el registro automático utilizando “sync_logs.insert_one()”, aquí se modificó el proceso de sincronización para almacenar fecha, base origen, base destino, cantidad de registros y control de errores.

- **pokedex/views.py**

```

from django.shortcuts import render
from services.mongo import sync_logs
def sync_logs_view(request):
    logs = sync_logs.find().sort(
        "date",
        -1
    )
    return render(
        request,

```

```

"sync_logs.html",
{
    "logs": logs
}
)

```

Se agrega la vista “sync_logs_view”, la cual se encarga de consultar y mostrar el historial guardado en la colección “sync_logs” de mongoDB, se puede consultar con la función find, y ordenar con la función sort, facilitando la visualización y el seguimiento de las sincronizaciones.

- **pokedex/urls.py**

```

from django.urls import path
from . import views
urlpatterns = [
    path(
        "sync_logs/",
        views.sync_logs_view,
        name="sync_logs"
    )
]

```

Se añade la ruta del historial para conectar la dirección “/sync_logs/” con la vista “sync_logs_view” para consultar y mostrar datos de sincronización desde mongoDB.

- **templates/sync_logs.html**

```

{% extends "base.html" %}
{% block content %}
<h2 class="mb-4">
    Historial de sincronizaciones
</h2>
<div class="table-responsive">
<table class="table table-dark table-striped">
<thead>
<tr>
<th>
Fecha
</th>

```

```

<th>
Origen
</th>
<th>
Destino
</th>
<th>
Registros
</th>
<th>
Estado
</th>
<th>
Errores encontrados
</th>
<th>
Errores solucionados
</th>
</tr>
</thead>
<tbody>
{% for log in logs %}
<tr>
<td>
{{ log.date }}
</td>
<td>
{{ log.origin }}
</td>
<td>
{{ log.destination }}
</td>
<td>
{{ log.records }}

```

```

</td>
<td>
{% if log.status == "SUCCESS" %}
<span class="badge bg-success">
Correcto
</span>
{% else %}
<span class="badge bg-danger">
Error
</span>
{% endif %}
</td>
<td>
{{ log.errorsFound }}
</td>
<td>
{{ log.errorsSolved }}
</td>
</tr>
{% empty %}
<tr>
<td colspan="7" class="text-center">
No existen registros de sincronización
</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% endblock %}

```

Se implementa una tabla para representar la información almacenada en “sync_logs”, ya con este cambio se puede consultar visualmente el historial, estados y resultados de sincronización del sistema.

10.3. Scripts.

Link de los scripts (Google drive):

<https://drive.google.com/drive/folders/1xj95mojZwk885Hzz2HRg6oe55OxsKkDR?usp=sharing>

10.4. Evidencias adicionales.

Link diagrama de componentes (Draw):

https://drive.google.com/file/d/1k_3ZF8MwSVDHyZ1n4ZXfsfPvh6edCegy/view?usp=sharing

Link diagrama de relaciones (Draw):

<https://drive.google.com/file/d/10SYU3NsCQfUUNdqI8e211AoVA9-Rg9fu/view?usp=sharing>

Link del repositorio (Github):

<https://github.com/PUCETEC-PROG2/laboratorio-django-Joffre1>

Link de los videos (Google drive):

https://drive.google.com/drive/folders/1amb8A-vCRWyWx_FgT7-BtORYHvHa87Wy?usp=sharing